

### GIFFLOWER

José Luis Vargas Rodríguez

### Documentación

Mayra Elizabeth Cervantes Ríos

Jazmín Alejandra Gutiérrez Morales

Hannia Junuen Jiménez Vázquez

Ángel Manuel Martínez Pastor

9° A - ISW

## Contenido

|                                     |   |
|-------------------------------------|---|
| Introducción .....                  | 3 |
| Objetivo general.....               | 3 |
| Objetivos específicos.....          | 3 |
| Requerimientos funcionales .....    | 3 |
| Requerimientos No Funcionales ..... | 4 |
| Diagramas.....                      | 4 |
| Pruebas postman .....               | 5 |
| Cotización/cálculo: .....           | 5 |
| HISTORIAL:.....                     | 8 |
| Excel: .....                        | 8 |

## Introducción

El proyecto GIFFLOWER consiste en desarrollar un sistema para automatizar el proceso de costeo de productos de una florería, sustituyendo el proceso manual realizado actualmente mediante la aplicación de hojas de cálculo, Excel.

Este sistema permitirá registrar las especies florales, administrar los diferentes factores involucrados en el cálculo del costo de venta y generar automáticamente el precio final de cada producto mediante reglas de negocio definidas

Además del cálculo automático el sistema almacena un historial de las operaciones realizadas y permitirá generar un archivo de hojas de cálculo individual por costeo, esto funcionará como un comprobante de la operación realizada.

La solución será implementada mediante una arquitectura basada en microservicios, esto buscar que el sistema sea escalable, mantenible y fácilmente extensible para incorporar nuevas funcionalidades a futuro.

## Objetivo general

Diseñar una arquitectura de software basada en microservicios que permitan automatizar el proceso de costeo de productos florales, garantizando modularidad, escalabilidad y facilidad de mantenimiento para futuras aplicaciones del sistema

## Objetivos específicos

- Diseñar una arquitectura realizada a base de microservicios
- Definir los componentes principales del sistema
- Modelar la interacción entre los servicios
- Establecer la comunicación entre los microservicios
- Diseñar la arquitectura de la base de datos No Relacional
- Definir los endpoints que expondrá cada servicio
- Establecer una estructura de desarrollo basada en buenas prácticas de desarrollo

## Requerimientos funcionales

- El sistema deberá permitir registrar especies florales
- El sistema permitirá modificar la información de cualquier especie registrada
- el sistema permitirá registrar nuevas flores
- el sistema deberá permitir modificar los factores para calcular el costo de venta
- el sistema deberá ejecutar de manera correcta todas las fórmulas de costeo
- el sistema deberá almacenar cada calculo registrado
- el sistema deberá generar un archivo de hojas de cálculo por cada calculo realizado
- el sistema deberá permitir actualizar parámetros generales para cambios de costos de operación

## Requerimientos No Funcionales

- El sistema será desarrollado en Node.js
- El código fuente será implementado con TypeScript
- La api rest será desarrollada con Express
- La información será almacenada en MongoDB
- La arquitectura seguirá el patrón de microservicios

## Diagramas

Diagrama de arquitectura general

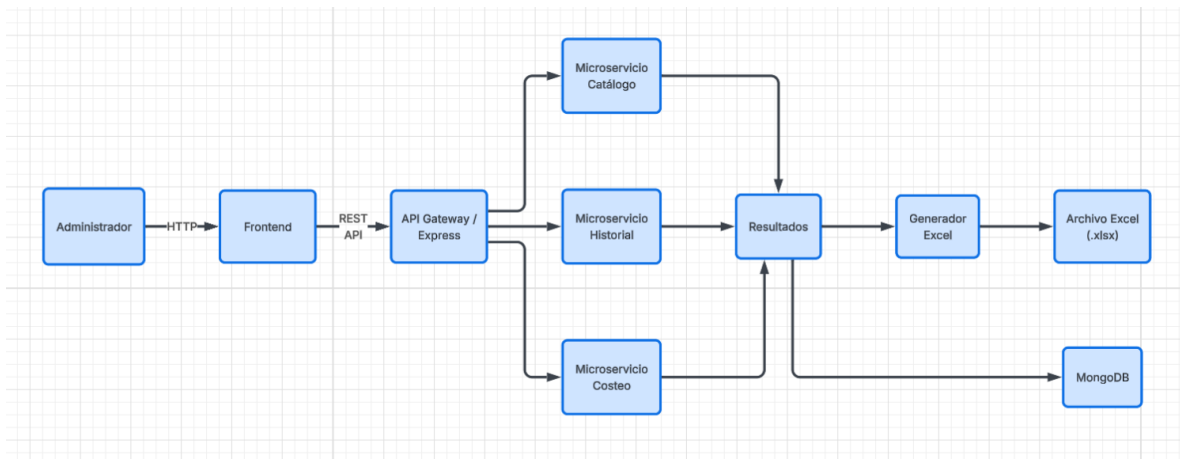


Diagrama de componentes

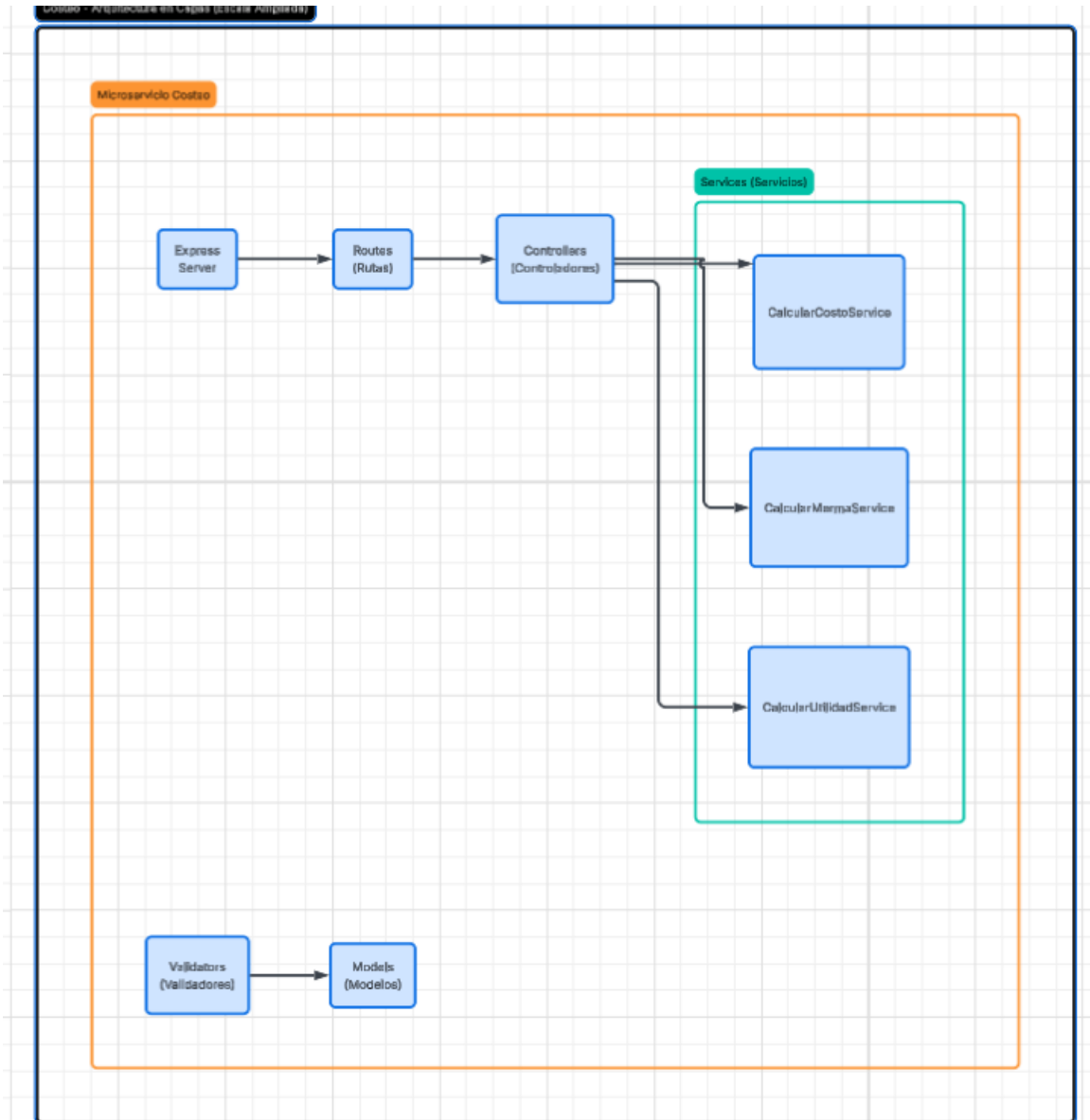


Diagrama de secuencia

## Pruebas postman

Cotización/cálculo:

Consulta1:

{

```
"descuento": 10,
"paquetes": [
  {
    "nombre": "Arreglo Primavera",
    "precioVenta": 500,
    "flores": [
      {
        "nombre": "Rosa",
        "cantidad": 10,
        "costoUnitario": 15
      },
      {
        "nombre": "Girasol",
        "cantidad": 5,
        "costoUnitario": 20
      }
    ]
  },
  {
    "nombre": "Arreglo Elegante",
    "precioVenta": 300,
    "flores": [
      {
        "nombre": "Clavel",
        "cantidad": 6,
        "costoUnitario": 12
      },
      {
        "nombre": "Tulipán",
```

```

    "cantidad": 4,
    "costoUnitario": 18
  }
]
}
]
}

```

#### Resultado:

Se realizó una petición POST al endpoint /cotizacion, enviando los datos de dos paquetes florales, incluyendo nombre, precio de venta y las flores utilizadas con su cantidad y costo unitario. El servicio calculó correctamente el costo de cada paquete, la utilidad, el subtotal, descuento y total de la cotización. La petición respondió con código HTTP 200.

```

Body    Cookies    Headers (7)    Test Results
Pretty  Raw    Preview    JSON v
1 [{"subtotal":394,"descuento":39.400000000000006,"total":354.6,"utilidadTotal":406,"paquetes":[{"nombre":"Arreglo Primavera","costoPaquete":250,"precioVenta":500,"utilidad":250},{nombre":"Arreglo Elegante","costoPaquete":144,"precioVenta":300,"utilidad":156}]}]

```

#### Consulta 2:

```

{
  "descuento": 10
}

```

#### Resultado:

Se realizó una prueba enviando una petición incompleta sin el arreglo de paquetes. El servidor detectó que no podía procesar la cotización y respondió con código HTTP 400, evitando registrar una operación inválida.

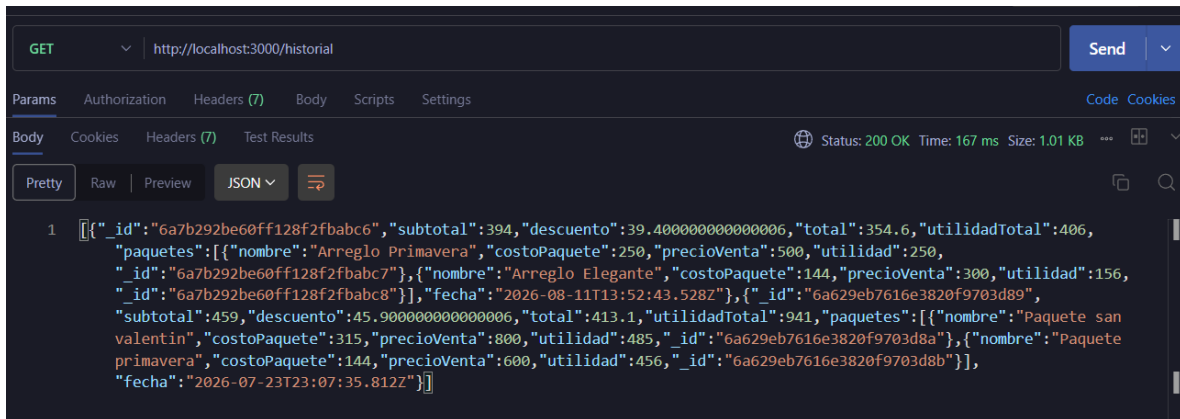
```

Body    Cookies    Headers (7)    Test Results
Pretty  Raw    Preview    JSON v
1 [{"mensaje":"Error al procesar la cotización.","detalle":"Cannot read properties of undefined (reading 'map')"}]

```

## Historial:

Se realizó una petición GET al endpoint /historial. El servidor consultó la colección de historial almacenada en MongoDB y regresó las cotizaciones registradas, ordenadas desde la más reciente. La petición respondió con código HTTP 200.



## Excel:

```
{  
  "descuento": 10,  
  "paquetes": [  
    {  
      "nombre": "Arreglo Primavera",  
      "precioVenta": 500,  
      "flores": [  
        {  
          "nombre": "Rosa",  
          "cantidad": 10,  
          "costoUnitario": 15  
        },  
        {  
          "nombre": "Girasol",
```



```

    "cantidad": 5,
    "costoUnitario": 20
  }
]
},
{
  "nombre": "Arreglo Elegante",
  "precioVenta": 300,
  "flores": [
    {
      "nombre": "Clavel",
      "cantidad": 6,
      "costoUnitario": 12
    },
    {
      "nombre": "Tulipán",
      "cantidad": 4,
      "costoUnitario": 18
    }
  ]
}
]
}

```

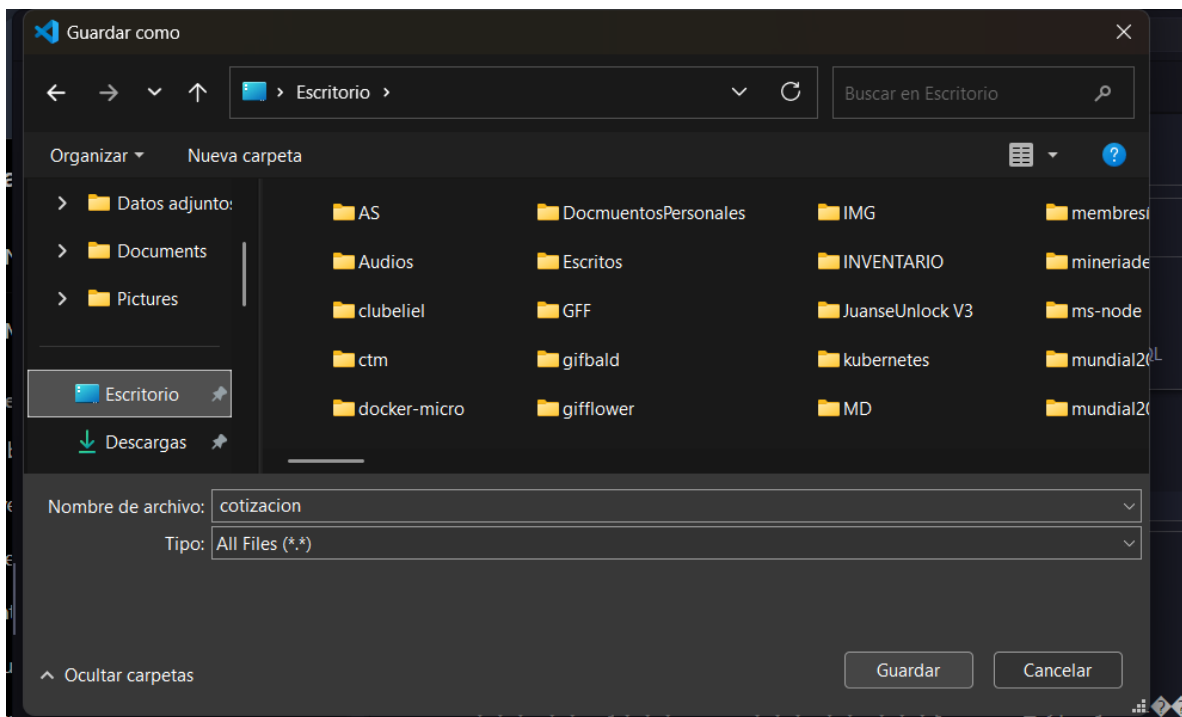
#### Resultado:

Se realizó una petición POST al endpoint /cotizacion/excel, enviando la información de una cotización. El servicio realizó los cálculos correspondientes y utilizó ExcelJS para generar dinámicamente un archivo .xlsx con los paquetes, costos, precios de venta, utilidad, subtotal, descuento y total.

```

Body    Cookies    Headers (8)    Test Results
Pretty  Raw    Preview    Text
1 PK
2 YB[Content_Types].xmlMnE=J]TUE4xB,w( 7{
m"mVqBsCn<y- 耶?IIE
3 P<ZT.4B6,r B,hcb[QX_e_ABBjrBB[BBwvBB<Ih+BBwBL
4 B
5 B/B)eB) ]UB+W
6 dBB16&BmBENL[F7BrBB}Bz}B0B+ggBn=5Bc@BBB) ]_Ii
7 SmBB<B
8 B]BB;1!B*=BBBb;+BBB[v?BPKB
9 BBAaq]BBBBBBBBBBB rels/PKB
10 BBAaq]BIKBKB rels/.relsBBB|EBnHB 4>$nBâB@BqBBB<BBB,
BBB*BBB#B2pBfL#JrYBHBzBBuB=MB+BBB0B1B)BBBvBBBtBzBBB51BB%_ln[oBBa
VB=BBHCBBBhBBBZBBuBB] BóKBBBB#wY c24'7|PKB
11 BBAaq]BBBBBBBBBBBxB1/PKB

```



|   | A                 | B            | C                   | D               | E |
|---|-------------------|--------------|---------------------|-----------------|---|
|   | <b>COTIZACIÓN</b> |              |                     |                 |   |
|   | <b>Paquete</b>    | <b>Costo</b> | <b>Precio Venta</b> | <b>Utilidad</b> |   |
|   | Arreglo Primavera | 250          | 500                 | 250             |   |
|   | Arreglo Elegante  | 144          | 300                 | 156             |   |
|   | Subtotal          | 394          |                     |                 |   |
|   | Descuento         | 39.4         |                     |                 |   |
|   | Total             | 354.6        |                     |                 |   |
| 0 | Utilidad Total    | 406          |                     |                 |   |
| 1 |                   |              |                     |                 |   |
| 2 |                   |              |                     |                 |   |
| 3 |                   |              |                     |                 |   |